

МИНОБРНАУКИ РОССИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ «ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ»

Факультет компьютерных наук
Кафедра программирования и информационных технологий

Лабораторная работа №1 по дисциплине «Архитектура информационных
систем»

Обучающийся

М.А.Ячный, 4 курс, группа 9.1

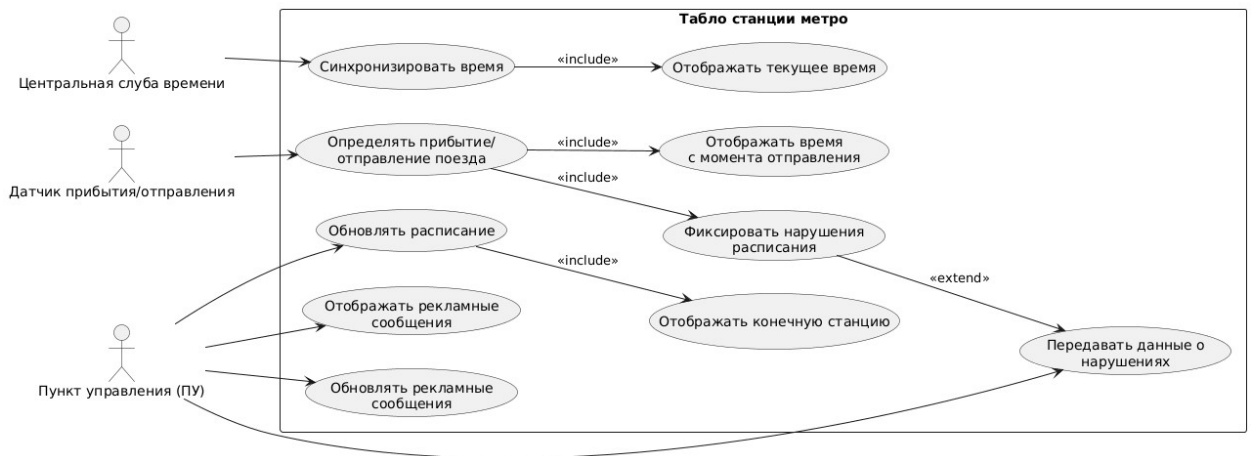
Руководитель

Е.В. Попова, старший преподаватель

Описание системы

Требуется разработать модель программного обеспечения «Табло для информационной службы метрополитена». Табло расположены на каждой станции метро. Они работают под управлением единого пункта управления (ПУ) информационной службы метро. Табло отображает текущее время (часы, минуты, секунды) и время, прошедшее с момента отправления последнего поезда (минуты, секунды). Момент прибытия и отправления поезда определяется при помощи датчиков, устанавливаемых на путях. Все табло метро синхронизованы, текущее время отсчитывается и устанавливается из центральной службы времени, находящейся на ПУ. На табло высвечивается конечная станция назначения прибывающего поезда. Эти данные содержатся в расписании движения поездов, которое хранится в памяти табло и периодически обновляется с ПУ. В «бегущей строке» табло отображается рекламная информация. Память табло хранит до 10 рекламных сообщений. Сообщения отображаются друг за другом с небольшими паузами, циклически. Содержание рекламных сообщений поступает с ПУ. Дополнительная функция табло – по запросу с ПУ оно пересылает данные о нарушениях расписания (преждевременных отправлениях поездов или опозданиях). В ходе выполнения задания должна быть создана схема базы данных для хранения рекламных сообщений, расписания и сведений о нарушении расписаний. Пояснение: в задании требуется разработать модель ПО только для табло, но не для пункта управления информационной службы.

Диаграмма вариантов использования



Код диаграммы

```

@startuml
left to right direction
actor "Пункт управления (ПУ)" as PU
actor "Датчик прибытия/отправления" as Sensor
actor "Центральная служба времени" as TimeService
rectangle "Табло станции метро" {
    usecase "Отображать текущее время" as UC1
    usecase "Синхронизировать время" as UC2
    usecase "Определять прибытие/отправление поезда" as UC3
    usecase "Отображать время\nc момента отправления" as UC4
    usecase "Отображать конечную станцию" as UC5
    usecase "Обновлять расписание" as UC6
    usecase "Отображать рекламные\nсообщения" as UC7
    usecase "Обновлять рекламные\nсообщения" as UC8
    usecase "Фиксировать нарушения\nрасписания" as UC9
    usecase "Передавать данные о\nнарушениях" as UC10
}
TimeService --> UC2
TimeService --> UC1
Sensor --> UC3
Sensor --> UC4
PU --> UC6
PU --> UC7
PU --> UC8
PU --> UC10
UC2 --> UC1 : <<include>>
UC3 --> UC4 : <<include>>
UC6 --> UC9 : <<include>>
UC7 --> UC5 : <<include>>
UC9 --> UC10 : <<extend>>
  
```

```
UC6 --> UC5 : <<include>>
UC3 --> UC9 : <<include>>
UC9 --> UC10 : <<extend>>
@enduml
```

Акторы

1. **Пункт управления (ПУ):** обновляет расписание движения поездов, передаёт рекламные сообщения и запрашивает данные о нарушениях
2. **Датчик прибытия/отправления:** передаёт сигналы о прибытии и отправлении поездов
3. **Центральная служба времени:** передаёт текущее время для синхронизации табло

Варианты использования

1.Отображать текущее время

Предусловие: табло синхронизировано с центральной службой времени

Основной поток:

1. Получение текущего времени
2. Отображение часов, минут, секунд

Постусловие: на табло отображается актуальное время

2.Синхронизировать время

Предусловие: установлено соединение с ПУ

Основной поток:

1. Получение сигнала синхронизации
 2. Обновление системного времени табло
- Постусловие: время табло синхронизировано

Связи: <<include>> → «Отображать текущее время»

3.Определять прибытие/отправление поезда

Предусловие: датчики исправны

Основной поток:

1. Получение сигнала от датчика
 2. Фиксация времени события
- Постусловие: зафиксирован факт прибытия/отправления

Связи:

- <<include>> → «Отображать время с момента отправления»
- <<include>> → «Фиксировать нарушения расписания»

4.Отображать время с момента отправления

Предусловие: зафиксировано отправление поезда

Основной поток:

1. Вычисление прошедшего времени
2. Отображение минут и секунд

Постусловие: отображается время ожидания следующего поезда

5.Отображать конечную станцию

Предусловие: расписание загружено

Основной поток:

1. Определение текущего поезда
2. Получение конечной станции из расписания
3. Вывод на табло

Постусловие: отображена конечная станция

6.Обновлять расписание

Предусловие: получен пакет обновления

Основной поток:

1. Получение данных расписания
 2. Проверка корректности
 3. Сохранение в памяти
- Постусловие: расписание обновлено

Связи: <<include>> → «Отображать конечную станцию»

7.Отображать рекламные сообщения

Предусловие: в памяти есть ≥ 1 сообщение

Основной поток:

1. Выбор сообщения
2. Отображение в «бегущей строке»
3. Пауза

4. Переход к следующему сообщению (циклически)
Постусловие: реклама отображается непрерывно

8.Обновлять рекламные сообщения

Предусловие: установлено соединение

Основной поток:

1. Получение новых сообщений (до 10)
2. Замена старых сообщений
Постусловие: рекламный блок обновлён

9.Фиксировать нарушения расписания

Предусловие: есть данные расписания и фактические события

Основной поток:

1. Сравнение фактического времени с плановым
2. Определение типа нарушения (опоздание/ранний уход)
3. Сохранение информации
Постусловие: нарушение зафиксировано

Связь: <<extend>> → «Передавать данные о нарушениях»

10.Передавать данные о нарушениях

Предусловие: получен запрос от ПУ

Основной поток:

1. Формирование отчёта
2. Передача данных в ПУ
Постусловие: данные о нарушениях отправлены

Диаграмма классов

Код диаграммы

```
@startuml
skinparam classAttributeIconSize 0
class DisplayController <<controller>> {
    - id : Long
    - currentTime : LocalDateTime
    - adManager : AdvertisementManager
    - violManager : ViolationManager
}
```

```

- timeService : TimeSyncService
- schedule : Schedule
+ updateTime(time : LocalDateTime) : void
+ showCurrentTime() : void
+ showElapsedTime(elapsed : Duration) : void
+ showDestination(station : String) : void
}

class Schedule <<entity>> {
- id : Long
- lastUpdate : LocalDateTime
- trainSchedules : List<TrainSchedule>
+ getTrainInfo(trainId : Long) : TrainSchedule
}

class TrainSchedule <<entity>> {
- id : Long
- trainId : Long
- destinationStation : String
- plannedArrival : LocalDateTime
- plannedDeparture : LocalDateTime
}

class Advertisement <<entity>> {
- id : Long
- text : String
- displayOrder : Integer
}

class AdvertisementManager <<service>> {
- maxAds : Integer
- ads: List<Advertisement>
+ rotateAds() : void
+ updateAds(ads : List<Advertisement>) : void
}

class TimeSyncService <<service>> {
+ getCurrentTime() : Long
}

class SensorEvent <<entity>> {
- id : Long
- sensorId : Long
- eventTime : LocalDateTime
}

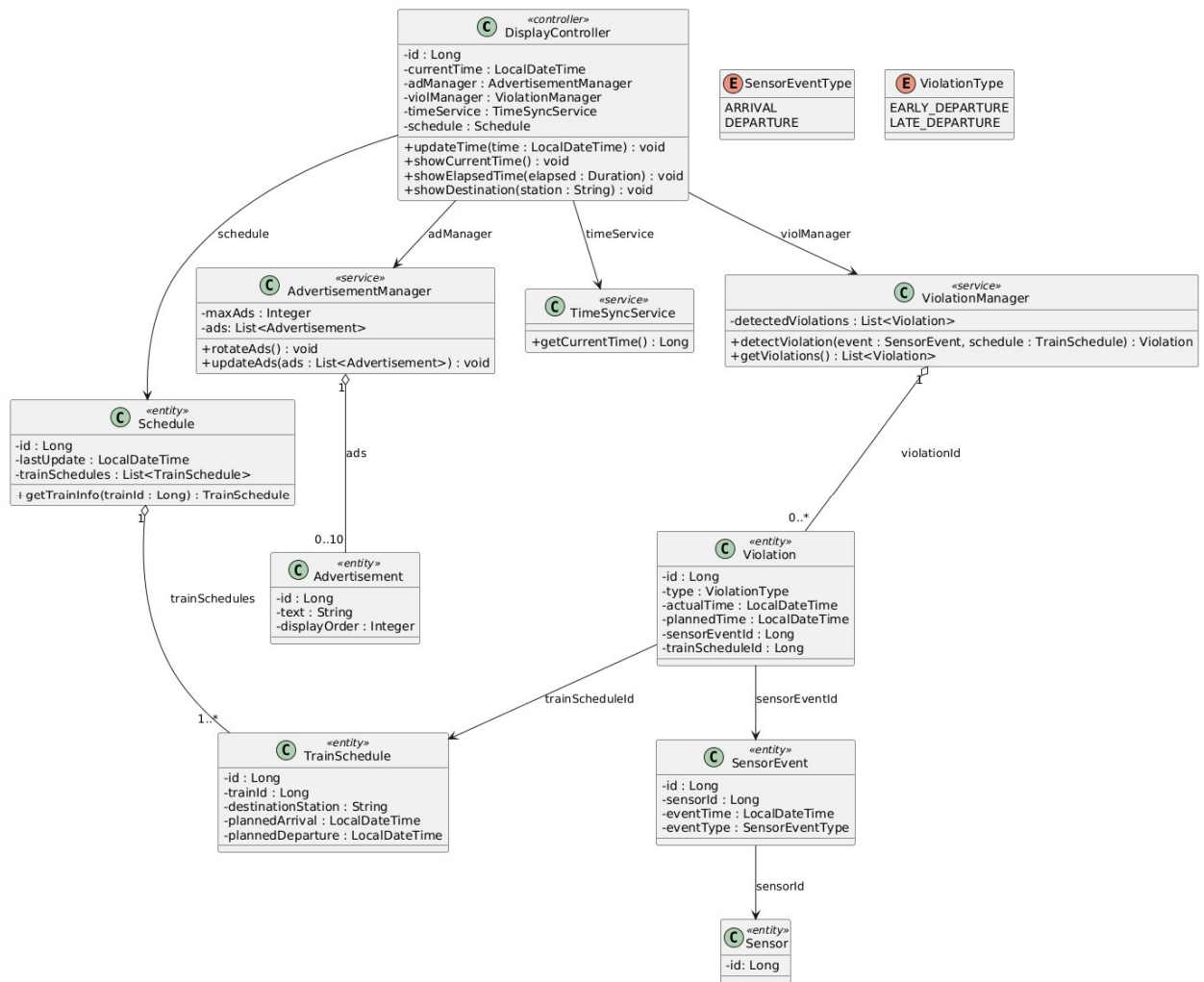
```

```

    - eventType : SensorEventType
}
class Sensor <<entity>> {
    - id: Long
}
enum SensorEventType {
    ARRIVAL
    DEPARTURE
}
class Violation <<entity>> {
    - id : Long
    - type : ViolationType
    - actualTime : LocalDateTime
    - plannedTime : LocalDateTime
    - sensorEventId : Long
    - trainScheduleId : Long
}
enum ViolationType {
    EARLY_DEPARTURE
    LATE_DEPARTURE
}
class ViolationManager <<service>> {
    - detectedViolations : List<Violation>
    + detectViolation(event : SensorEvent, schedule : TrainSchedule) :
Violation
    + getViolations() : List<Violation>
}
Schedule "1" o-- "1..*" TrainSchedule : trainSchedules
AdvertisementManager "1" o-- "0..10" Advertisement : ads
ViolationManager "1" o-- "0..*" Violation : violationId
DisplayController --> Schedule : schedule
DisplayController --> AdvertisementManager : adManager
DisplayController --> TimeSyncService : timeService
DisplayController --> ViolationManager : violManager
Violation --> TrainSchedule : trainScheduleId
Violation --> SensorEvent : sensorEventId
SensorEvent --> Sensor : sensorId
@enduml

```


Изображения диаграммы



DisplayController <<controller>>

Назначение: управление отображением информации на табло.

Поля:

- id : Long
- currentTime : LocalDateTime – Текущее время
- adManager : AdvertisementManager – Менеджер рекламы
- violManager : ViolationManager – Менеджер нарушений
- timeService : TimeSyncService – Сервис синхронизации времени
- schedule : Schedule – Текущее расписание

Операции:

- updateTime(time : LocalDateTime) : void – установить новое время
- showCurrentTime() : void – отобразить время

- showElapsedTime(elapsed : Duration) : void – отобразить время с момента отбытия поезда
- showDestination(station : String) : void – отобразить конечную станцию поезда

Связи:

- зависимость от Schedule через поле schedule
- зависимость от AdvertisementManager через поле adManager
- зависимость от TimeSyncService через поле timeService
- зависимость от ViolationManager через поле violManager

Schedule <<entity>>

Назначение: хранение расписания движения поездов.

Поля:

- id : Long
- lastUpdate : LocalDateTime – Последнее время обновления
- trainSchedules : List<TrainSchedule> - Расписания конкретных поездов

Операции:

- getTrainInfo(trainId : Long) : TrainSchedule – получить информацию о конкретном поезде

Связи:

- агрегация (aggregation) с TrainSchedule: кратность: 1 ко многим, через поле schedules

TrainSchedule <<entity>>

Назначение: данные одного поезда.

Поля:

- id : Long
- trainId : Long – id поезда
- destinationStation : String – Конечная станция
- plannedArrival : LocalDateTime – Запланированное время прибытия
- plannedDeparture : LocalDateTime – Запланированное время отбытия

Advertisement <<entity>>

Назначение: рекламное сообщение.

Поля:

- id : Long
- text : String – Содержание
- displayOrder : Integer – Приоритет отображения

AdvertisementManager <<service>>

Назначение: управление рекламными сообщениями.

Поля:

- maxAds : Integer – Максимальное количество реклам (в данной реализации 10)
- ads : List <Advertisement> - Список реклам

Операции:

- rotateAds() : void – Отображать рекламы
- updateAds(ads : List<Advertisement>) : void – Получить обновление реклам

Связи:

- агрегация с Advertisement, кратность: 1 - 0..10, через поле advertisementId

TimeSyncService <<service>>

Назначение: синхронизация времени табло.

Операции:

- getCurrentTime() : получить актуальное текущее время

SensorEvent <<entity>>

Назначение: событие от датчика.

Поля:

- id : Long
- eventTime : LocalDateTime

- eventType : SensorEventType

Violation <<entity>>

Назначение: нарушение расписания.

Поля:

- id : Long
- type : ViolationType
- actualTime : LocalDateTime
- plannedTime : LocalDateTime

Связи:

- зависимость от TrainSchedule через trainScheduleId
- зависимость от SensorEvent через sensorEventId

ViolationManager <<service>>

Назначение: обнаружение и хранение нарушений.

Поля:

- id : Long
- violations : List <Violation> - Засечённые нарушения

Операции:

- detectViolation(event : SensorEvent, schedule : TrainSchedule) : void - сформировать и сохранить засечённое нарушение
- getViolations() : List<Violation> - получить все нарушения

Связи:

- агрегация с Violation кратность: 1 - 0..* через поле violations

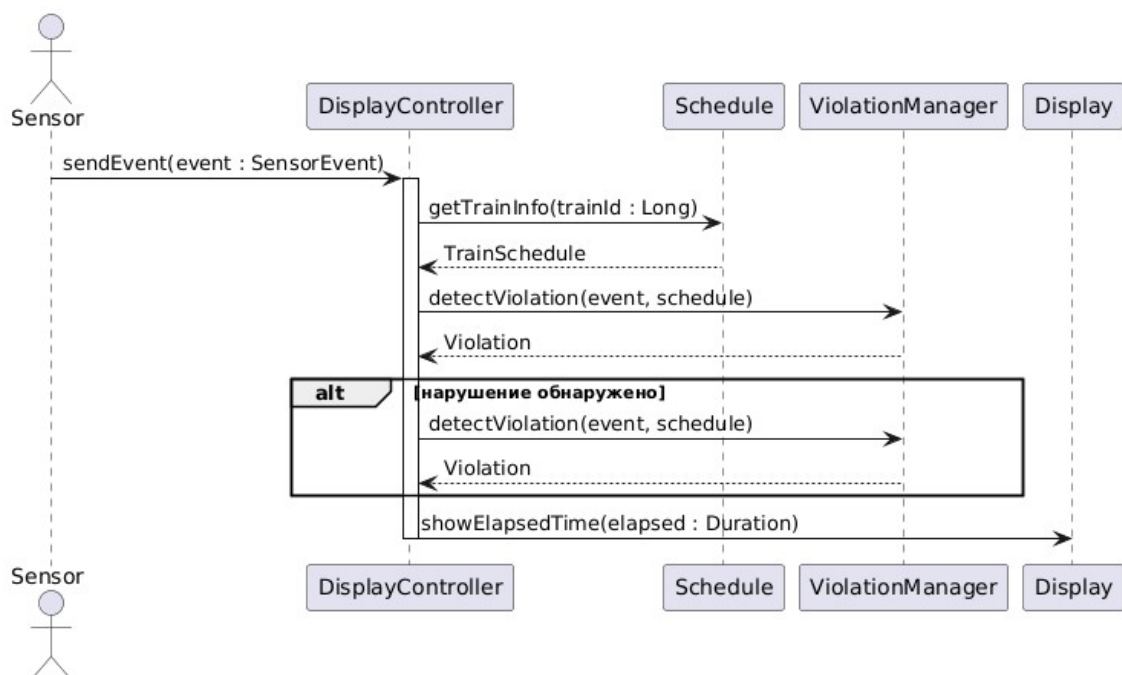
Диаграммы последовательностей

Код диаграммы 1 (Получение сигнала с сенсора и фиксация нарушений)

```
@startuml
actor Sensor
participant "DisplayController" as DC
participant "Schedule" as SCH
participant "ViolationManager" as VM
participant "Display" as UI

Sensor -> DC : sendEvent(event : SensorEvent)
activate DC
DC -> SCH : getTrainInfo(trainId : Long)
SCH --> DC : TrainSchedule
DC -> VM : detectViolation(event, schedule)
VM --> DC : Violation
alt нарушение обнаружено
    DC -> VM : detectViolation(event, schedule)
    VM --> DC : Violation
end
DC -> UI : showElapsedTime(elapsed : Duration)
deactivate DC
@enduml
```

Изображение диаграммы 1:



Объекты:

- Sensor — источник события
- DisplayController – менеджер рекламы
- Schedule – расписание
- ViolationManager – менеджер нарушений
- Display (UI) – экран

Сообщения:

1. sendEvent(event : SensorEvent)
2. getTrainInfo(trainId : Long) : TrainSchedule
3. detectViolation(event : SensorEvent, schedule : TrainSchedule) : Violation
4. showElapsedTime(elapsed : Duration) : void

Управляющая информация:

- Используется комбинированный фрагмент alt
- Условие: обнаружено ли нарушение расписания
- При наличии нарушения создаётся объект Violation

Результат сценария:

- Вычислено время с момента отправления
- При необходимости зафиксировано нарушение

Код диаграммы 2 (Обновление рекламных сообщений)

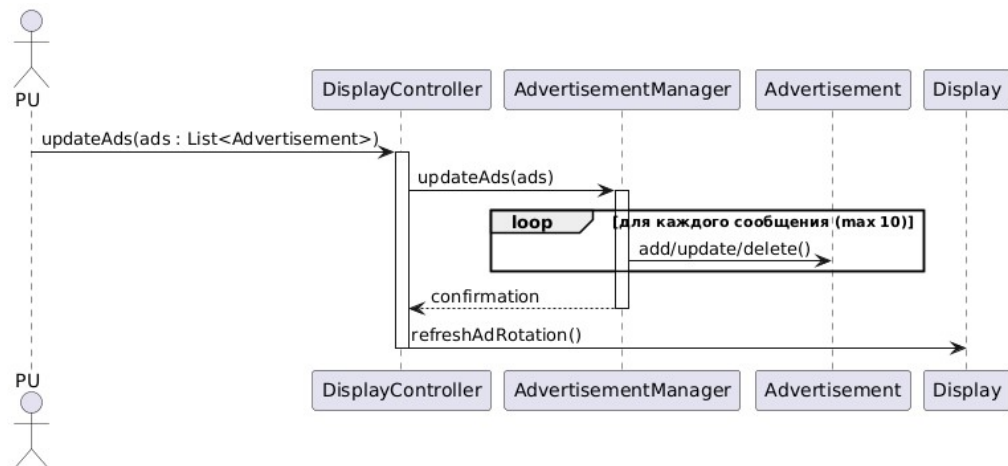
```
@startuml
actor PU
participant "DisplayController" as DC
participant "AdvertisementManager" as AM
participant "Advertisement" as AD
participant "Display" as UI
PU -> DC : updateAds(ads : List<Advertisement>)
activate DC
DC -> AM : updateAds(ads)
activate AM
loop для каждого сообщения (max 10)
    AM -> AD : create/update
end
```

```

AM --> DC : confirmation
deactivate AM
DC -> UI : refreshAdRotation()
deactivate DC
@enduml

```

Изображение диаграммы 2



Объекты:

- PU — внешний объект
- DisplayController – контроллер табло
- AdvertisementManager – менеджер рекламы
- Advertisement – реклама
- Display – экран

Сообщения:

1. updateAds(ads : List<Advertisement>)
2. create/update/delete()
3. refreshAdRotation()

Управляющая информация:

- Используется цикл loop
- Ограничение: не более 10 рекламных сообщений
- После обновления запускается повторная ротация

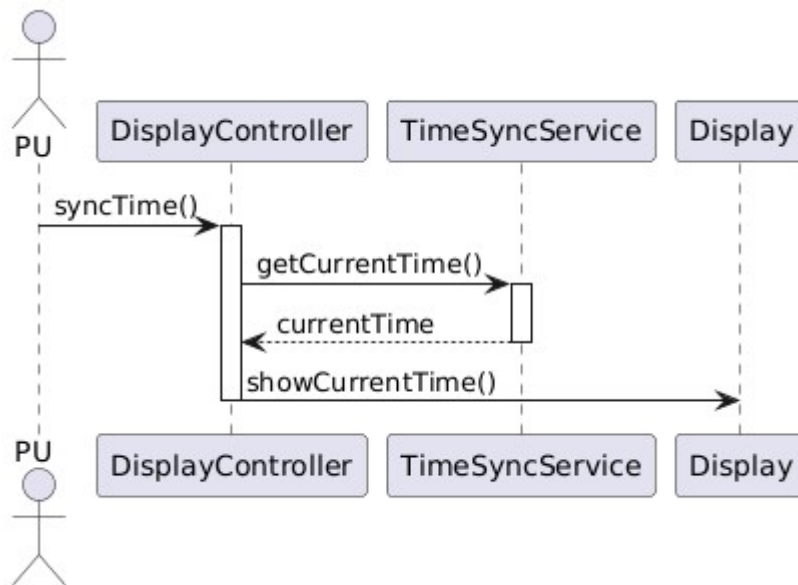
Результат:

- Рекламные сообщения обновлены
- Запущен новый цикл отображения

Код диаграммы 3 (Синхронизация времени)

```
@startuml
actor PU
participant "DisplayController" as DC
participant "TimeSyncService" as TS
participant "Display" as UI
PU -> DC : syncTime()
activate DC
DC -> TS : getCurrentTime()
activate TS
TS --> DC : currentTime
deactivate TS
DC -> UI : showCurrentTime()
deactivate DC
@enduml
```

Изображение диаграммы 3



Объекты:

- PU – пульт управления
- TimeSyncService – сервис синхронизации времени
- DisplayController – контроллер табло
- Display – экран

Сообщения:

1. syncTime(time : LocalDateTime)
2. updateTime(time : LocalDateTime)
3. showCurrentTime()

Результат:

- Системное время обновлено
- На табло отображается актуальное время

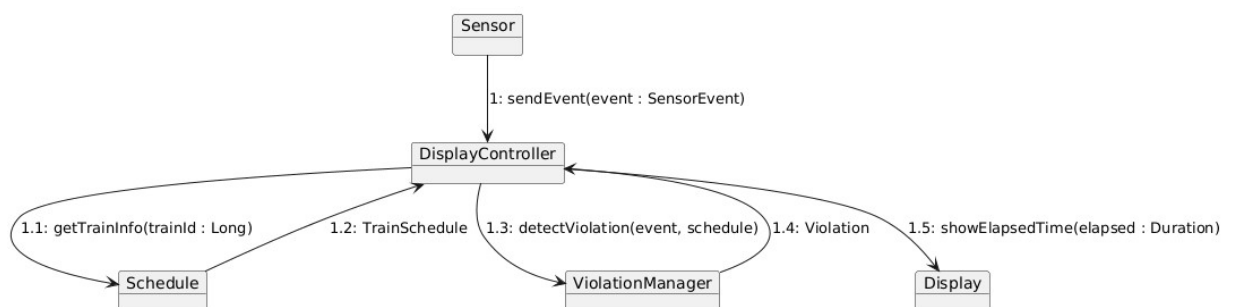
Диаграммы кооперации

Код диаграммы 1 (Получение сигнала с сенсора и фиксация нарушений)

```
@startuml
object Sensor
object DC as "DisplayController"
object SCH as "Schedule"
object VM as "ViolationManager"
object UI as "Display"

Sensor --> DC : 1: sendEvent(event : SensorEvent)
DC --> SCH : 1.1: getTrainInfo(trainId : Long)
SCH --> DC : 1.2: TrainSchedule
DC --> VM : 1.3: detectViolation(event, schedule)
VM --> DC : 1.4: Violation
DC --> UI : 1.5: showElapsedTime(elapsed : Duration)
@enduml
```

Изображение диаграммы 1:



Объекты:

- Sensor – Сенсор
- DisplayController – контроллер табло
- Schedule – расписание
- ViolationManager – менеджер нарушений

- Display – экран табло

Пронумерованные сообщения:

1. sendEvent(event : SensorEvent) инициируется датчиком

1.1 getTrainInfo(trainId : Long) : TrainSchedule – запрос объекта расписания поезда

1.2 возврат объекта TrainSchedule

1.3 detectViolation(event : SensorEvent, schedule : TrainSchedule) – обнаружение и формирование объекта нарушения

1.4 возврат объекта Violation

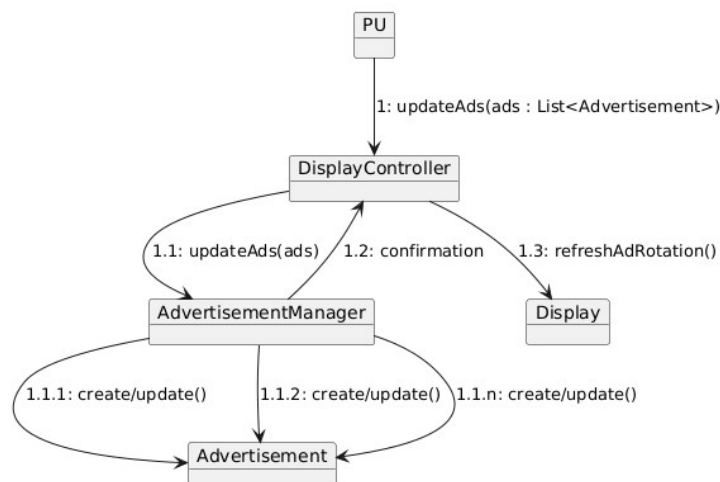
1.5 showElapsedTime(elapsed : Duration) – отображение текущего времени

Код диаграммы 2 (Обновление рекламы)

```
@startuml
object PU
object DC as "DisplayController"
object AM as "AdvertisementManager"
object AD as "Advertisement"
object UI as "Display"

PU --> DC : 1: updateAds(ads : List<Advertisement>)
DC --> AM : 1.1: updateAds(ads)
AM --> AD : 1.1.1: create/update()
AM --> AD : 1.1.2: create/update()
AM --> AD : 1.1.n: create/update()
AM --> DC : 1.2: confirmation
DC --> UI : 1.3: refreshAdRotation()
@enduml
```

Изображение диаграммы 2:



Объекты:

- PU – пульт управления
- DisplayController – контроллер табло
- AdvertisementManager – менеджер рекламы
- Advertisement – реклама
- Display – экран табло

Пронумерованные сообщения:

1.updateAds(ads : List<Advertisement>) – сообщение от пульта управления

1.1 updateAds(ads : List<Advertisement>) – запрос к менеджеру рекламы

1.1.1-n create/update() – для каждого рекламного сообщения (не более 10)

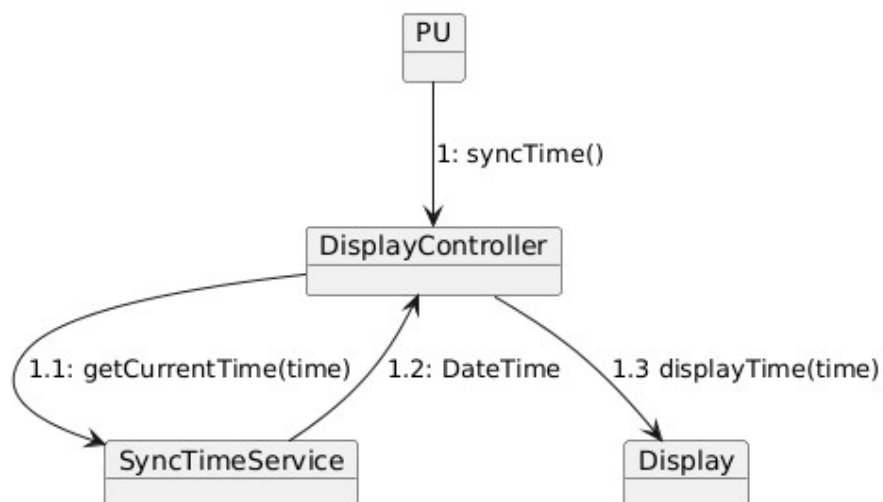
1.2 подтверждение обновления

1.3 refreshAdRotation() : void – обновление ротации рекламы на дисплее

Код диаграммы 3 (Синхронизация времени)

```
@startuml
object PU as "PU"
object TS as "SyncTimeService"
object DC as "DisplayController"
object UI as "Display"
PU --> DC : 1: syncTime()
DC --> TS : 1.1: getCurrentTime(time)
TS --> DC : 1.2: DateTime
DC --> UI : 1.3 displayTime(time)
@enduml
```

Изображение диаграммы 3:



Объекты:

- PU – Пункт управления
- TimeSyncService – Сервис синхронизации времени
- DisplayController – Контроллер табло
- Display – Экран табло

Пронумерованные сообщения:

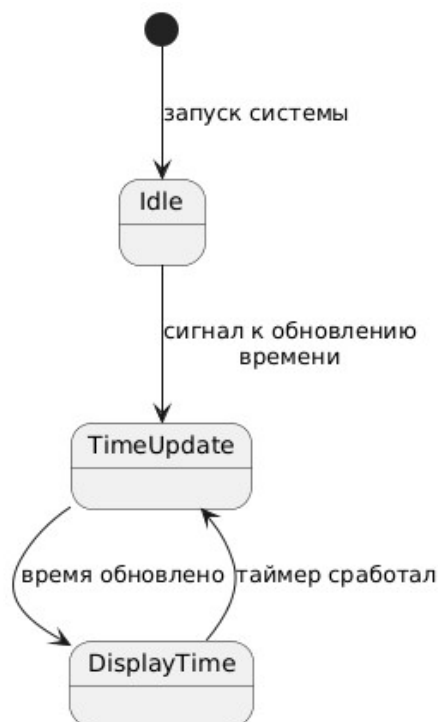
- 1.syncTime() – инициализация синхронизации времени
- 1.1 getCurrentTime() – запрос текущего времени
- 1.2 возврат объекта DateTime
- 1.3 displayTime(time) – отображение текущего времени на табло

Диаграммы состояний

Код диаграммы 1 (Обновление времени)

```
@startuml
[*] --> Idle : systemStart
Idle --> TimeUpdate : событие= syncTime(time)
TimeUpdate --> DisplayTime : условие = время обновлено
DisplayTime --> TimeUpdate : условие = таймер сработал
@enduml
```

Изображение диаграммы 1:



Idle

- Деятельность: Ожидание сигнала синхронизации
- Входное действие: -
- Выходное действие: -
- Событие: Сигнал к обновлению времени

TimeUpdate

- Деятельность: Получение нового времени
- Входное действие: Запрос на получение времени от SyncTimeService
- Выходное действие: Сохранение нового времени
- Событие: Получение текущего времени от SyncTimeService

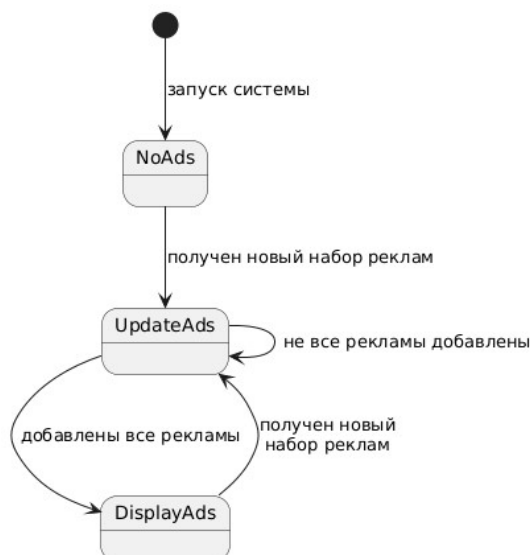
DisplayTime

- Деятельность: Отображение текущего времени
- Входное действие: Вывод текущего времени на табло
- Выходное действие: -
- Событие: сработал таймер необходимости снова обновить время

Код диаграммы 2 (Обновление рекламы)

```
@startuml
[*] --> NoAds : запуск системы
NoAds --> UpdateAds: получен новый набор реклам
DisplayAds --> UpdateAds : получен новый\nнабор реклам
UpdateAds --> UpdateAds : не все рекламы добавлены
UpdateAds --> DisplayAds : добавлены все рекламы
@enduml
```

Изображение диаграммы 2:



NoAds

- Деятельность: Ожидание получения первых реклам
- Входное действие: -
- Выходное действие: -
- Событие: Получение нового набора реклам от ПУ

UpdateAds

- Деятельность: Добавление очередной рекламы из набора в список
- Входное действие: Добавить рекламу в список
- Выходное действие: Проверить, остались ли еще рекламы
- Событие: Добавлены все рекламы

DisplayAds

- Деятельность: Отображение циклически всех реклам
- Входное действие: Начать вывод обновлённых реклам
- Выходное действие: -
- Событие: Получение нового набора реклам от ПУ

Код диаграммы 3 (Обработка сигналов с сенсоров)

```
@startuml
[*] --> WaitingForEvent : запуск системы
WaitingForEvent --> EventReceived : получен\нсигнал
EventReceived --> WaitingForEvent : нарушения не\нобнаружено
EventReceived --> ViolationDetected : нарушение\нобнаружено
ViolationDetected --> WaitingForEvent : нарушение\ндобавлено
@enduml
```

Изображение диаграммы 3:



WaitingForEvent

- Деятельность: Ожидание сигнала от одного из датчиков
- Входное действие: -
- Выходное действие: -
- Событие: Получение сигнала от датчика

EventReceived

- Деятельность: Обработка сигнала и сверка с расписанием
- Входное действие: Обновить данные на табло (время после отбытия/конечная станция прибывающего поезда)
- Выходное действие: Сверить время прибытия/отбытия поезда с расписанием
- Событие: Определено нарушение/не нарушение

ViolationDetected

- Деятельность: Создание объекта нарушения и добавление его в список
- Входное действие: Создать объект нарушения и добавить в список
- Выходное действие: -
- Событие: Объект нарушения добавлен

Диаграммы активностей

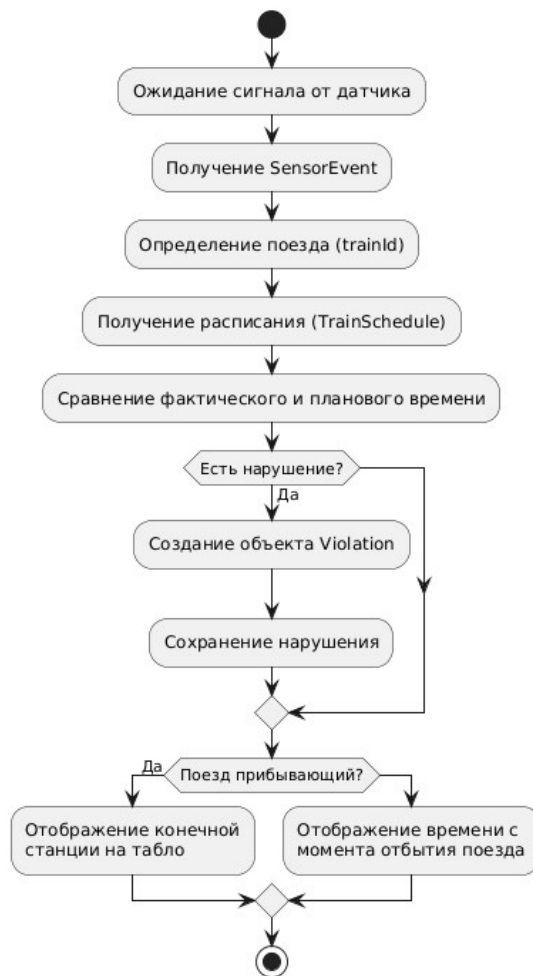
Код диаграммы 1 (Обработка сигналов с сенсоров)

```
@startuml
start
:Ожидание сигнала от датчика;
:Получение SensorEvent;
:Определение поезда (trainId);
:Получение расписания (TrainSchedule);
:Сравнение фактического и планового времени;
if (Есть нарушение?) then (Да)
    :Создание объекта Violation;
    :Сохранение нарушения;
endif
if (Поезд прибывающий?) then (Да)
    :Отображение конечной\нстанции на табло;
else
    :Отображение времени с\нмомента отбытия поезда;
endif
```

stop

@enduml

Изображение диаграммы 1:



Состояния деятельности:

1. Ожидание сигнала от датчика
2. Получение события SensorEvent
3. Определение поезда
4. Получение расписания
5. Сравнение времени
6. Создание нарушения (при необходимости)
7. Вычисление прошедшего времени
8. Отображение информации

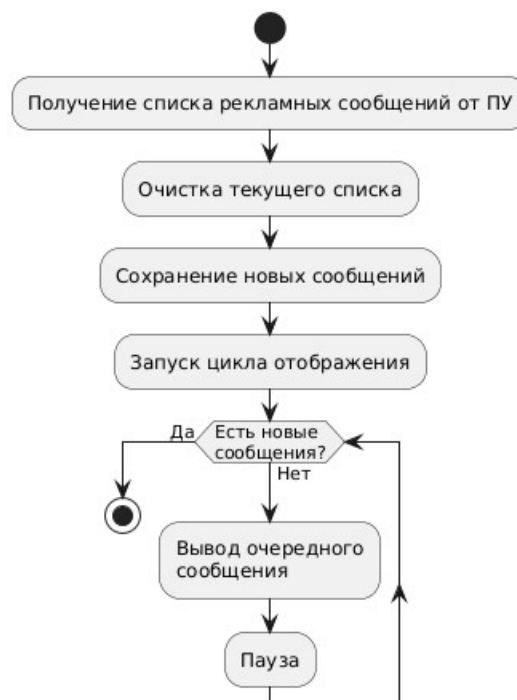
Условия:

1. Есть ли нарушение?
2. Является ли поезд прибывающим?

Код диаграммы 2 (Отображение рекламы)

```
@startuml
start
:Получение списка рекламных сообщений от ПУ;
:Очистка текущего списка;
:Сохранение новых сообщений;
:Запуск цикла отображения;
while (Есть новые\nсообщения?) is (Нет)
    :Вывод очередного\nсообщения;
    :Пауза;
endwhile (Да)
stop
@enduml
```

Изображение диаграммы 2:



Состояния деятельности:

1. Получение сообщений
2. Очистка старых сообщений
3. Сохранение новых сообщений
4. Запуск цикла отображения
5. Вывод сообщения
6. Пауза

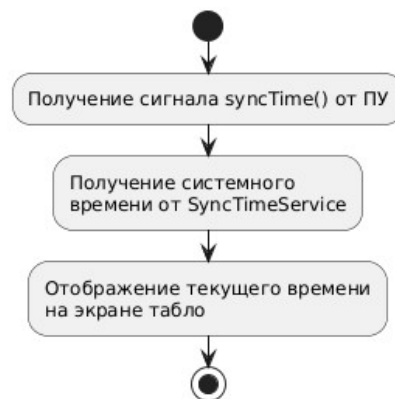
Условия:

1. Получены ли во время отображения новые сообщения?

Код диаграммы 3 (Синхронизация времени)

```
@startuml
start
:Получение сигнала syncTime() от ПУ;
:Получение системного\nвремени от SyncTimeService;
:Отображение текущего времени\nна экране табло;
stop
@enduml
```

Изображение диаграммы 3:



Состояния деятельности:

1. Получение сигнала синхронизации
2. Обновление системного времени
3. Отображение текущего времени на табло

Диаграмма компонентов

Код диаграммы

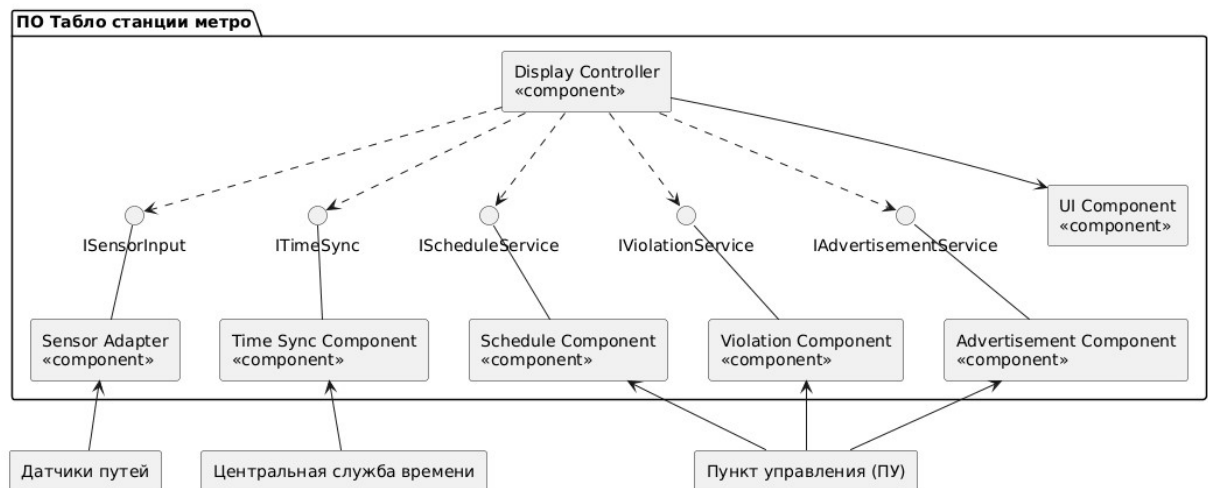
```
@startuml
skinparam componentStyle rectangle
package "ПО Табло станции метро" {
    component "UI Component\n<<component>>" as UI
    component "Display Controller\n<<component>>" as DC
    component "Time Sync Component\n<<component>>" as TS
    component "Schedule Component\n<<component>>" as SCH
    component "Advertisement Component\n<<component>>" as ADV
}
```

```

component "Violation Component\n<<component>>" as VIOL
component "Sensor Adapter\n<<component>>" as SENS
interface ITimeSync
interface IScheduleService
interface IAdvertisementService
interface IViolationService
interface ISensorInput
TS -u- ITimeSync
SCH -u- IScheduleService
ADV -u- IAdvertisementService
VIOL -u- IViolationService
SENS -u- ISensorInput
DC ..> ITimeSync
DC ..> IScheduleService
DC ..> IAdvertisementService
DC ..> IViolationService
DC ..> ISensorInput
DC --> UI
}

component "Пункт управления (ПУ)" as PU
component "Центральная служба времени" as CTS
component "Датчики путей" as SENSOR_EXT
PU -u-> ADV
PU -u-> SCH
PU -u-> VIO
CTS -u-> TS
SENSOR_EXT -u-> SENS
@enduml

```



1. Display Controller Component

Назначение: центральная координация работы табло.

Спецификация:

- Управление отображением
- Координация сервисов
- Обработка входящих событий

Зависимости:

- Зависит от интерфейсов:
 - ITimeSync
 - IScheduleService
 - IAdvertisementService
 - IViolationService
 - ISensorInput
- Использует UI Component

2. UI Component

Назначение: вывод информации на экран табло.

Спецификация:

- Отображение времени
- Отображение конечной станции
- Отображение рекламы
- Отображение времени ожидания

Зависимости:

- Используется Display Controller

3. Time Sync Component

Назначение: синхронизация времени табло.

Предоставляемый интерфейс: ITimeSync

Функции:

- Приём сигнала syncTime
- Обновление системного времени

Зависимости:

- Получает данные от Центральной службы времени

4. Schedule Component

Назначение: хранение и обработка расписания поездов.

Предоставляемый интерфейс: IScheduleService

Функции:

- Хранение расписания
- Поиск информации о поезде
- Обновление расписания

Зависимости:

- Обновляется ПУ

5. Advertisement Component

Назначение: управление рекламными сообщениями.

Предоставляемый интерфейс: IAdvertisementService

Функции:

- Хранение до 10 сообщений
- Циклическая ротация
- Обновление данных

Зависимости:

- Получает данные от ПУ

6. Violation Component

Назначение: фиксация нарушений расписания.

Предоставляемый интерфейс: IViolationService

Функции:

- Сравнение фактического и планового времени
- Создание записи о нарушении
- Передача данных по запросу ПУ

Зависимости:

- Использует данные Schedule Component

- Получает запросы от ПУ

7. Sensor Adapter

Назначение: приём сигналов от датчиков путей.

Предоставляемый интерфейс: ISensorInput

Функции:

- Приём сигналов ARRIVAL / DEPARTURE
- Формирование событий для системы

Зависимости:

- Получает сигналы от физических датчиков

Внешние компоненты:

- Пункт управления – Обновление расписания, рекламы, запрос нарушений
- Центральная служба времени – Передача текущего времени
- Датчики/Сенсоры путей – Передача событий прибытия/отправления

Диаграмма размещения

Код диаграммы:

```
@startuml
skinparam componentStyle rectangle
node "Пункт управления (ПУ)\n<<device>>" as PU {
    artifact "Система управления\nрасписанием и рекламой"
}
node "Центральная служба времени\n<<device>>" as CTS {
    artifact "Time Server"
}
node "Станция метро\n<<location>>" {
    node "Контроллер табло\n<<device>>" as BOARD {
        artifact "ПО
Табло\n(DisplayController,\nSchedule,\nAdvertisement,\nViolation,\nTimeSync,\nSensorAdapter)"
    }
    node "Датчики путей\n<<device>>" as SENSOR {
```

```

    artifact "Sensor Hardware"

}

node "Экран табло\n<<device>>" as SCREEN {
    artifact "Display Firmware"

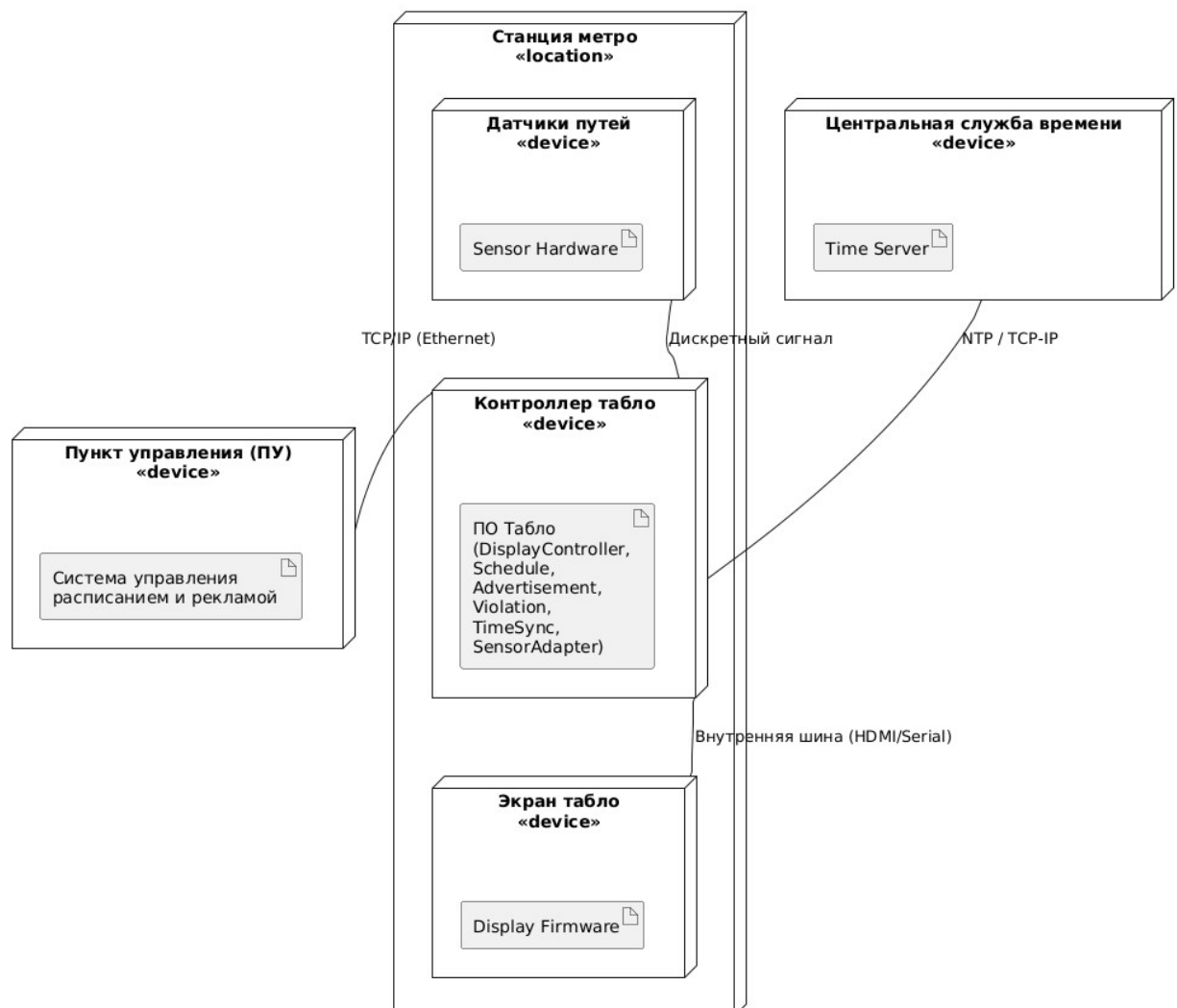
}

}

PU -r- BOARD : TCP/IP (Ethernet)
CTS -- BOARD : NTP / TCP-IP
SENSOR -- BOARD : Дискретный сигнал
BOARD -- SCREEN : Внутренняя шина (HDMI/Serial)
@enduml

```

Изображение диаграммы:



1. Пункт управления (ПУ) <<device>>

Назначение:

- передача расписания,

- передача рекламных сообщений,
- запрос информации о нарушениях.

Размещённые артефакты:

- Система управления расписанием и рекламой.

Связи:

- TCP/IP соединение с контроллером табло.

2. Центральная служба времени <<device>>

Назначение:

- передача сигнала синхронизации времени.

Размещённые артефакты:

- Time Server.

Связи:

- NTP / TCP-IP соединение с контроллером табло.

3. Станция метро <<location>>

Физическая площадка размещения оборудования.

3.1 Контроллер табло <<device>>

Основной вычислительный узел системы.

Размещённые программные компоненты:

- DisplayController
- Schedule Component
- Advertisement Component
- Violation Component
- TimeSync Component
- Sensor Adapter

Связи:

- Получает данные от ПУ
- Получает время от Центральной службы времени
- Получает сигналы от датчиков
- Передаёт данные на экран табло

3.2 Датчики путей <<device>>

Назначение:

- фиксация прибытия и отправления поездов.

Связь:

- передача дискретного сигнала на контроллер табло.

3.3 Экран табло <<device>>

Назначение:

- физическое отображение информации.

Связь:

- подключение к контроллеру по внутренней шине (HDMI/Serial).

Соединения и взаимосвязи:

Откуда	Куда	Тип соединения	Назначение
ПУ	Контроллер табло	TCP/IP	Передача расписания и рекламы
Центральная служба времени	Контроллер табло	NTP	Синхронизация времени
Датчики	Контроллер табло	Дискретный сигнал	Передача событий
Контроллер табло	Экран	Внутренняя шина	Отображение данных